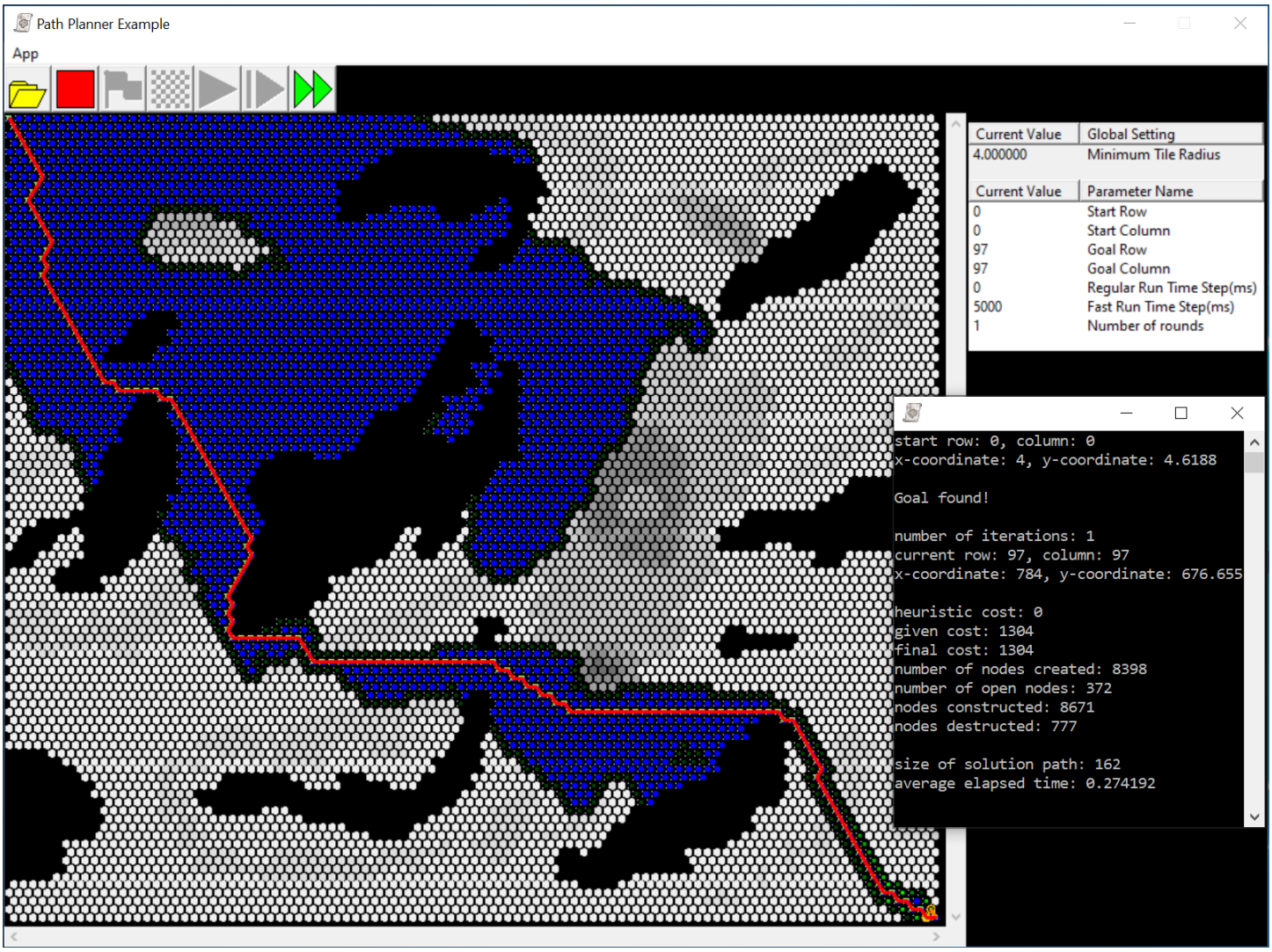
Manual: Path Planner

Overview

The tile-based **Path Planner** application allows for creation, demonstration, and analysis of different search algorithms using techniques common to tile-based simulations. The user of the finished product can...

- Configure the algorithm via a parameter control panel.
- Incorporate tile painting to visualize the current state of searches. and
- Use step-wise, tick (delta-time) based, and completion run modes to pinpoint progress.

Students will implement a search algorithm with the dual-goals of efficiency in memory/time and optimal pathing i***n terms of cost***. The UI code and drawing interface are provided to students; students are expected to implement the search itself by conforming to this specification.



Objectives & Outcomes

Upon completion of this activity, students should be able to...

- Implement, trace, debug, and optimize a search algorithm class using a standardized API
- Work with graph-based structures, including traversing and searching within those data structures

Development

The default (and supported) way to use this lab is to use **Visual Studio Code** with the **Extension Pack for Java** (by Microsoft) installed. The project also requires **Java Development Kit 11** or higher. Finally, you will need to download and install Qt (https://www.qt.io/download-qt-installer-oss) and QtJambi (https://www.qtjambi.io).

For QtJambi, it can either be manually hunted down (https://search.maven.org/artifact/io.qtjambi/qtjambi/), or links can be constructed based on your OS, architecture, and Qt version:

Main file: https://repo1.maven.org/maven2/io/qtjambi/qtjambi/{VER}
Platform file: https://repo1.maven.org/maven2/io/qtjambi/qtjambi-native-{OS}/{VER}

Here, {VER} is the version of Qt (e.g., **6.9.2**), and {OS} is one of "**macos**", "**linux-x64**", "**linux-arm64**", "**windows-x64**", or "**windows-arm64**". Place the base and native jar files in the "lib" folder of the project, then open the folder in VS Code.

Structure

It is recoomend the complete the project as a sequence of several smaller steps:

1) Creating a search graph to represent the **`TileMap`**, which in turn represents the terrain.
2) Writing of a declaration for the **`PathSearch`** class and initial (likely empty) methods for compilation.
3) Development of a visual debugging strategy using the drawing functions of the **`TileMap`**.
4) Implementation of a Breadth-First Search, then modifying it into a Greedy Best-First Search.
5) Modification of the Greedy Best-First Search into a Uniform-Cost Search, and then into an A* Search.
6) Optimization of the A* algorithm to meet speed / benchmark requirements.

User Interface (UI)

The UI for the path planner is provided for students so that they may focus on implementing the search algorithm itself within the existing code base (as is common in the game and other industry sectors). Upon launch, the planner will load the default map and run the search algorithm's initialization routine. For example, the search implementation might build and display a search graph upon initialization (**Figure 1**).

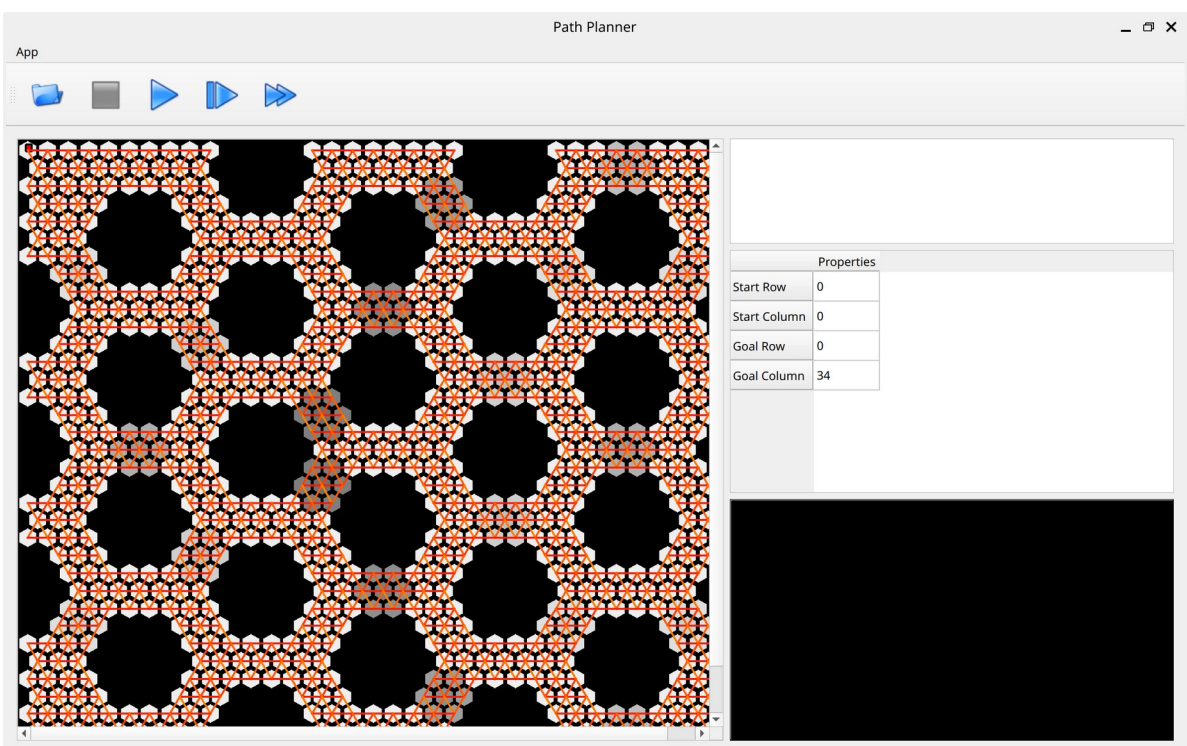


Figure 1. Path planner application upon initial load, displaying a search graph built by the search algorithm.

Tile Map Display

The tiles that represent terrain are displayed, along with any drawing engaged by the search algorithm, in the Tile Map Display (bottom-left). Heavier terrain weights are represented by darker tiles, while black tiles represent untraversable terrain. The red flag represents the starting point and the green flag the goal location.

Button Controls

The Button Controls across (top) the user to control the search as identified (**Table 1**).

Icon	Name	Description
	Open Tile Map	Brings up file dialog. Once you select a file, all memory allocated to the old tile map is freed, a new tile map is loaded from that file, all search algorithms are reset, and all node counters are zeroed out.
	Reset Search	Resets current search algorithm & updates node counters. If all search algorithms have been reset this way, then each pair of constructed/destructed counts should match.
	Run / Pause	When enabled and currently paused, runs the search algorithm until it reaches the goal tile, animating its progress in the meantime. If pressed while the search algorithm is still running, it becomes paused. As soon as the search algorithm reaches the goal tile or can make no further progress, this button will be disabled.
	Step	When enabled, runs search algorithm for exactly one iteration and then pauses it. As soon as the search algorithm reaches the goal tile or can make no further progress, this button will be disabled.
	Time Run	Resets search algorithm, then runs search algorithm until it reaches a goal tile; **does not animate progress**. The number of iterations shown is replaced by elapsed time from initialization to finish.

Table 1. Search Application Controls

Settings Pane

Several options that can be changed in ***Settings*** (upper-right), include setting start & goal locations and time limits for single step or full algorithm runs. For full runs, the number of iterations can be adjusted to measure time performance. *Note: minimum tile radius is for display only; it cannot be changed.*

Development Interface

The development interface includes several elements to help students test and debug their algorithms.

Console

The console window automatically spawns and can be written to using standard streams (e.g., `cout`).

Drawing System

Several debugging functions & methods are provided to draw to the screen, including filling tiles, drawing markers, setting outlines, and drawing lines.

	Lum	Red	Grn	Blue	Hex
	255	0	255	0	0xFF00FF00
	127	0	255	0	0x7F00FF00
	255	127	0	0	0xFF7F0000
	0	0	0	255	0x000000FF

Table 2. Example colors in the LRGB color model.

The color space for these drawing functions is 32-bit LRGB space (**luminosity**, **red**, **green**, and **blue**.) Each color is a 32-bit number with the first (high-order) byte holding luminosity (**Table 2**, **Figure 4**).

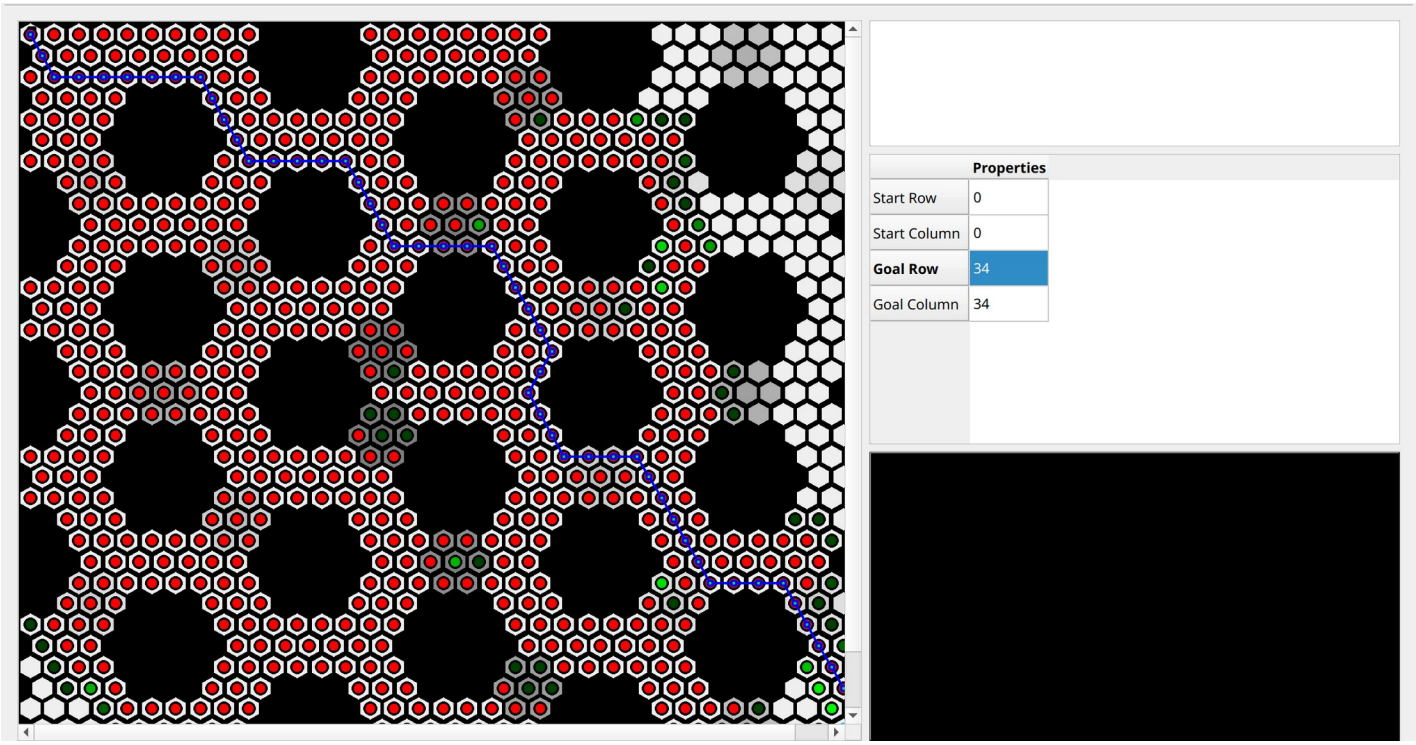


Figure 4. Example of colors in use in the Path Planner application.

State Space

Some engines (like this planner) use hexagonal tiles, which are equidistant from their neighbors (unlike square tiles, which have a different distance to neighbors in cardinal directions vs diagonal neighbors). This simplifies & speeds up calculations, but it makes storing and retrieving tiles in a typical 2D array less straightforward. An example is outlined in **Figure 2**. Each tile also holds a terrain weight which identifies the cost of traversing through the tile.

Array	**Column**			
Row	0	1	2	3
0	X	✔	✔	-
1	✔		✔	-
2	X	✔	✔	-
3	-	✔	✔	X
4	-	✔		✔
5	-	✔	✔	X

Figure 2. Layout of hexagonal grids (rows and columns).

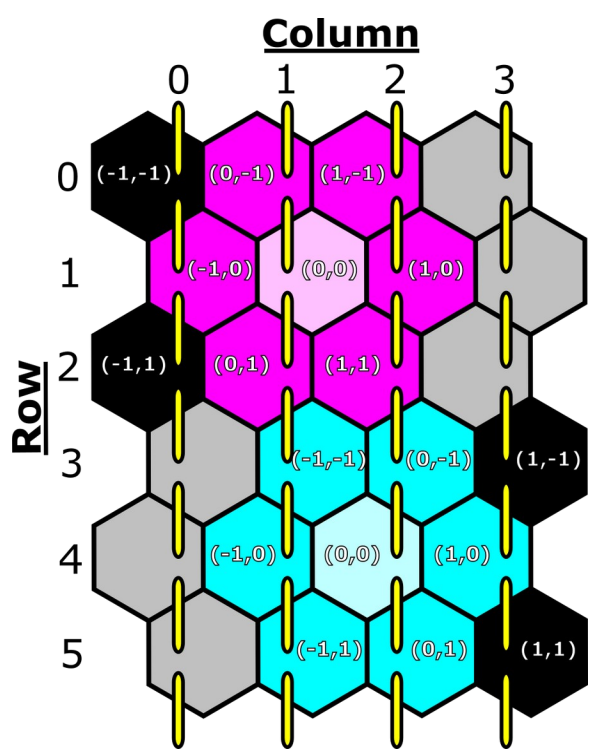


Figure 3. How tile information is stored in an array.

In **Figure 3**, the dashed yellow lines separate the tiles into logical columns. The offset from the current tile (0, 0) to its neighbors is marked; odd rows have offsets as outlined **MAGENTA** and even rows have offsets as outlined in **CYAN**. **BLACK** tiles indicate offsets that are invalid depending on the odd or even row location of (0,0). **Note that these are not the same as the obstacle tiles in the tile map**. We can see how these data would be stored in a 2D array (**Figure 3**). For any tile whose row coordinate has an odd value, the tiles that are adjacent to it follow the pattern of the magenta tiles. For any tile whose row coordinate has an even value, the tiles that are adjacent to it follow the pattern of the cyan tiles.

It is recommended that students build a private helper method in their search class to determine the adjacency of `Tile` objects to simplify identification of neighboring locations, e.g.:

```
private boolean areAdjacent(Tile lhs, Tile rhs)
```

Developer API (Provided)

The following pinterface is provided so students can implement the search algorithm. Note: students **may not change** the API provided; student searches must function properly without any API changes.

TileSystem

These classes are in the **`edu.uwu.tiles`** package and the **`Framework/TileSystem`** directory.

TileMap

`public Tile getTile(int row, int column)`
Returns a pointer to the `Tile` at the designated **`row`** and **`column`**.

`public int getRowCount()`
Returns the number of rows of tiles in this map.

`public int getColumnCount()`
Returns the number of columns of tiles in this map.

`public double getTileRadius()`
Returns the length of the radius of tiles in this map.

`public void resetTileDrawing()`
Resets (wipe) all drawing on the tiles in this map.

Tile

`public int getRow()`
Returns this `Tile`'s row in the map.

`public int getColumn()`
Returns this `Tile`'s column in the map.

`public int getWeight()`
Returns this `Tile`'s terrain weight.

`public double getX()`
Returns this `Tile`'s x-axis coordinate on the map.

`public double getY()`
Returns this `Tile`'s y-axis coordinate on the map.

`public int getFill()`
Returns this `Tile`'s LRGB fill color.

`public int getMarker()`
Returns this *Tile*'s LRGB marker color.

`public int getOutline()`
Returns this *Tile*'s LRGB outline color.

`public ArrayList<Line> getLines()`
Returns an `ArrayList` of lines being drawn from this *Tile*.

`public void setFill(int color)`
Sets this *Tile*'s fill color to **color** in the 32-bit LRGB space (luminosity, red, green, and blue).

`public void setMarker(int color)`
Sets this *Tile*'s marker color to **color** in the 32-bit LRGB space (luminosity, red, green, and blue).

`public void setOutline(int color)`
Sets this *Tile*'s outline color to **color** in the 32-bit LRGB space (luminosity, red, green, and blue).

`public void addLineTo(Tile endpoint, int color)`
Sets up a line to be drawn from this *Tile* to **endpoint** using the designated **color**.

`public void clearLines()`
Removes all lines being drawn from this *Tile*.

`public void resetDrawing()`
Resets all drawing on this tile.

NOTE: users **should not** create tiles or tile maps; instead, they should use references to existing objects.

General

This class resides within the **edu.uwu.data** package and in the root of the source directory.

PriorityQueue<T>
`public PriorityQueue(Comparator<T> comparator)`
Constructor for container that holds type *T*. Parameter is a Java **comparator**; that can compare objects of type *T*, indicating which is greater (or if they are equal.) See example below for lambda-based shortcut!

`public bool isEmpty()`
Returns **true** if the container has no elements in it, and **false** otherwise.

`public size_t size()`
Returns the number of elements currently in the container.

`public ArrayList<T> enumerate()`
Returns a **sorted** container containing all elements from greatest to least.

`public void push(T element)`
Pushes **element** into the container; the container maintains sorting / ordering during this operation.

`public T pop()`
Removes and returns the lowest value (“top”) element from the container.

`public void remove(const T &element)`
Removes all instances of **element** from the container.

`public void clear()`
Removes all elements from the container.

Figure 5 is example of how the *PriorityQueue* type might be used and function in code:

```
import java.util.ArrayList;
import edu.uwu.data.PriorityQueue;

public class Mine
{
 static int compare(Integer l, Integer r) {return l – r;}

 public static void main(String[] args)
 {
  PriorityQueue<Integer> pQ;
  pQ = new PriorityQueue<>((lhs,rhs) → compare(lhs,rhs));

  pQ.push(4);
  pQ.push(2);
     pQ.push(3);

   ArrayList<Integer> output = pQ.enumerate();

  for (Integer value : output)
     System.out.println(value);
 }
}
```

Figure 5a. Example of PriorityQueue class use in code.

```
4
3
2
–
```

Figure 5b. Example Output

NOTE: As ***PriorityQueue***'s constructor requires a parameter, to create a ***PriorityQueue*** object as a member variable within another class, it will need to be initialized **when the object is created**!

Assignment API (TODO)

Students will complete the *PathSearch* class according to the specification outlined here exactly. This class **must reside** within the **edu.uwu.student** package.

PathSearch

public **PathSearch**()
The constructor; takes no arguments.

public void **load**(TileMap _tileMap)
Called after the tile map is loaded. This is usually where the search graph is generated. ***NOTE***: This method must take no longer than twice the benchmark example for any given map.

public void **initialize**(int startRow, int startCol, int goalRow, int goalCol)
Called before any update of the path planner; should prepare for search to be performed between the tiles at the coordinates indicated. This method is always preceded by at least one call to **load()**.

public void **update**(long timeslice)
Called to allow the path planner to execute for the specified **timeslice** (in milliseconds). Within this method, the search should be performed until time expires or solution is found. If **timeslice** is zero (0), this method should only do a single iteration of the algorithm. Otherwise, it should only iterate for the indicated number of milliseconds. This method is always preceded by at least one call to **initialize()**.

public void **shutdown**()
Called when the current search data is no longer needed. It should clean up any memory allocated for this search. Note that this is **not** the same as the destructor, as the search object may be reset to perform another search on the same map. May be called before **initialize()** and/or **update()** in some cases.

public void **unload**()
Called when tile map is unloaded; should clean up any memory allocated for ile map. This is not the same as the destructor, as the search object may be reinitialized with a new map. May be called before **load()**.

public boolean **isDone()**
Returns **true** if the update function has finished because it found a solution, and **false** otherwise. Once a search has completed, this method should continue to return true until **initialize()** is called.

public ArrayList<Tile> **getSolution()**
Return an *ArrayList* containing solution path as an ordered series of *Tile* pointers from finish to start. Once a search has been completed, this method continues to return the path until **initialize()** is called.

Execution Frames
As the algorithm must be split across multiple execution frames, students will need to identify in which method tasks will be accomplished. Below is one example of how a search might be constructed in this way.

load() – Create search graph from the tile map by identifying adjacent tiles.
initialize() – Set starting & ending points for search, adding the starting point to the search queue.
update() – Run body of algorithm, looping over queue, until time is expired, saving state for next run.
shutdown() – Reset search's member variables so that another search on the same terrain can be run.
unload() – Free all memory associated with the tile map (i.e., the search graph).

Submissions

Students will submit a **zip file** containing the following files at the end of this exercise on Canvas:

- PathSearch.java

Place them in the **root directory** of your zip file, not in a subdirectory. **Do not submit any other files.**

Sample Tests

It is recommended that students test using these samples as well as developing a list of **their own unit tests and test cases** in order to thoroughly test their algorithm. Please note that projects will be tested using separate, unique data that follows the specification, so self-testing is crucial to proper function.

Tile Map	Start		Goal	
	Row	Col.	Row	Col.
hex035x035.txt	31	19	3	15
hex035x035.txt	0	9	34	26
hex035x035.txt	2	19	31	19
hex035x035.txt	17	32	17	2
hex035x035.txt	32	32	6	0
hex035x035.txt	17	3	17	31
hex054x045.txt	44	53	3	51
hex054x045.txt	30	53	30	0
hex054x045.txt	22	0	22	53
hex054x045.txt	3	51	42	53
hex054x045.txt	8	0	34	53
hex054x045.txt	22	51	22	53
hex054x045.txt	2	51	44	53

Tile Map	Start		Goal	
	Row	Col.	Row	Col.
hex098x098.txt	97	97	0	0
hex098x098.txt	38	44	97	0
hex098x098.txt	3	51	90	40
hex098x098.txt	52	0	53	97
hex098x098.txt	50	7	41	97
hex098x098.txt	93	0	53	80
hex098x098.txt	5	92	69	52
hex113x083.txt	82	112	0	0
hex113x083.txt	0	16	82	97
hex113x083.txt	14	0	70	112
hex113x083.txt	81	73	1	15
hex113x083.txt	41	3	41	109